

Appendix – I

Appendix A

Initial Meeting October 13, 2024

Student: Hey Isabel, thanks for meeting with me. So, let's talk about this playlist program, what exactly are you looking for?

Client: Hi Yeah, so basically, I want something that helps me create playlists, but I don't want some that just throws in random songs. I want to be able to create new playlists with music I already like with new recommendations.

Student: Okay, so you want it to be more hands-on, like you're in control but with some guidance?

Client: Exactly. Like, if I'm making a study playlist, I want to pull in songs I know work for me but also have suggestions that fit the vibe.

Student: That makes sense. I can set it up where you can browse music by genre, artist, album, and pick what you want. I can also add a way to shuffle in songs similar to what you've chosen.

Client: Yeah, that sounds perfect. Also, can I save multiple playlists and name them?

Student: Yep, you'll be able to name them, edit them, and save them all in one file. Is there anything else you would want the program to do?

Client: Maybe a way to see all the songs I've added so I don't accidentally pick the same ones?

Student: Yeah it's doable. I'll keep track of that in the interface. I'll start building out the base structure and check in with you later once there's something to test.

Client: Sounds good. Thank you for doing this.

Music Recommendations Interview November 25, 2024

Student: Alright, music time. I need a list of your favorite artists, albums, or anything you'd want in your playlists.

Client: Oh, this is my moment. Okay, definitely One Direction, Sabrina Carpenter, The Wallows, Conan Gray, Clairo, Djo, Declan McKenna.

Student: Love that. I'll add those into the program so you can easily pick from them. Any specific albums or songs you love from each?

Client: Oh yeah, for The Wallows, *Nothing Happens* is a must. And I NEED "Are You Bored Yet?" with Clairo in there.

Student: I got you. What about genres?

Client: Mostly indie pop, alt rock, and some old-school pop. Basically, chill but not too slow.

Student: Sweet, I'll make sure it is easy to navigate. Anything you DON'T want?

Client: Yeah, NO country, please. And nothing too electronic-heavy, it kinda distracts me when studying.

Student: Okay I'll check in once I have the music selection set up.

Progress Check-in January 10, 2025

Student: Hey Isabel, so I've got most of the program working. You can create playlists, browse by genre and artist, and even shuffle in new songs.

Client: That's so cool. So I can actually name my playlists now?

Student: Yep, you name it, pick your songs, and it saves everything. You can also go back and edit later.

Client: Have you tested if it keeps the playlists saved even after closing?

Student: Everything is stored in a file, so even if you exit, your playlists will still be there.

Client: That's great. The only thing I'd want to check is if I can remove songs easily, like if I add one by accident.

Student: I'll add a "remove song" option so you can change your list before saving. Anything else?

Client: Not that I can think of. Can't wait to try it out.

Black Box Testing March 3, 2025

Student: Alright, Isabel, today's the day. You get to test the program without me telling you how it works, just dive in and let me know if anything feels off.

Client: Ooh, okay.

Client: I can see my playlists, navigate through artists, but, how do I go back if I pick the wrong genre?

Student: I can add a back button to make navigation easier. Anything else?

Client: Yeah, when I pick a song, it jumps back to the genre menu, but I kinda expected it to stay on the album so I could add more.

Student: Makes sense. I'll change it so it keeps you in the album until you're done picking.

Client: Love that. Other than that I love it. It actually feels easy to use.

Final Meeting March 7, 2025

Student: So, final thoughts, how's it working for you?

Client: Honestly, it's awesome. It does exactly what I wanted, lets me mix my music!

Student: That's the goal. Anything you wish was different?

Client: Hmm... maybe in the future, a way to auto-suggest songs based on what's in my playlist? Like a "people who like this also like" kind of thing and maybe a easier way to be able to put it on music platforms?

Student: Ooh, that's a great idea. That would be the next step,

Client: But yeah, overall, I love it. Thanks for making this.

Student: Glad you like it. Let me know if you ever want any changes in the future.

Appendix C

```
package Music;
import java.util.Scanner;
public class main {
    public static void main(String[] args) {
        System.out.println("Welcome to MusicMixer");
        System.out.println("What would you like to do?");
        Scanner scanner = new Scanner(System.in);
        Create objC = new Create(); // Create an instance of Create to access the songs
        while (true) {
            System.out.println("\n1. View a playlist\n2. Generate a playlist\n3. Create a
playlist\n4. Clear current playlist\n5. Remove song from playlist\n6.Queue a song\n7. Show
Queue\n8.Exit");
            String decision = scanner.nextLine();
            if (decision.equals("1")) { // View existing playlists
                View.displayPlaylists();
            }
            else if (decision.equals("2")) { // Generate a playlist
                System.out.print("How many songs would you like in your generated playlist?
");
                int playlistLength = Integer.parseInt(scanner.nextLine());
                Generate objB = new Generate(objC); // Pass the Create instance to Generate
                objB.generatePlaylist(playlistLength); // Generate the playlist
            }
            else if (decision.equals("3")) { // Create a new playlist
                objC.createPlaylist(); // Use the existing instance of Create
            }
            else if (decision.equals("4")) { // Clear current playlist
                objC.clearPlaylist();
            }
            else if (decision.equals("5")) { // Remove song from playlist
                objC.removeSongFromPlaylist();
            }
            else if (decision.equals("6")) { // Queue a song
                objC.queueSong();
            }
            else if (decision.equals("9")) { // ignore this one, removed from GUI
                objC.playNextSong();
            }
            else if (decision.equals("7")) { // Show Queue
                objC.showQueue();
            }
            else if (decision.equals("8")) { // Exit the program
                System.out.println("Exiting the program. Goodbye!");
                break; // Exit the loop
            }
            else {
                System.out.println("Invalid input, please try again.");
            }
        }
    }
}
```

```

        scanner.close();
    }
}

```

```
package Music;
```

```

import java.nio.file.Files;
import java.nio.file.Paths;
import java.io.IOException;
import java.util.Collections;
import java.util.List;
import java.util.ArrayList;

```

```

public class Generate {
    private Create create; // Reference to the Create instance

    public Generate(Create create) {
        this.create = create; // Initialize the Create instance
    }

    public void generatePlaylist(int playlistLength) {
        List<String> availableSongs = getAllAvailableSongs(); // Get all available songs from the Create
        Collections.shuffle(availableSongs); // Shuffle

        List<String> generatedPlaylist = new ArrayList<>(); // Create a list to hold the generated playlist

        System.out.println("Generated Playlist:");

        for (int i = 0; i < Math.min(playlistLength, availableSongs.size()); i++) {
            generatedPlaylist.add(availableSongs.get(i)); // Add to the generated playlist
            System.out.println("- " + availableSongs.get(i));
        }

        savePlaylistToFile(generatedPlaylist); // Save the playlist to a file
    }

    private List<String> getAllAvailableSongs() {
        List<String> allSongs = new ArrayList<>();
        for (Genre genre : create.getMusicLibrary().values()) {
            for (Artist artist : genre.getArtists().values()) {
                for (Album album : artist.getAlbums().values()) {
                    for (Song song : album.getSongs()) {
                        allSongs.add(song.getTitle());
                    }
                }
            }
        }
    }
}

```

```

    }
}
return allSongs; // Return the list of all available songs
}

private void savePlaylistToFile(List<String> playlist) {
    try {
        Files.write(Paths.get("playlists.txt"), playlist); // Write the playlist to playlists.txt
        System.out.println("Playlist saved to playlists.txt");
    } catch (IOException e) {
        System.err.println("Error saving playlist: " + e.getMessage());
    }
}
}

```

```

package Music;
import java.util.Collections;
import java.util.ArrayList;
import java.util.Scanner;
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.io.File;
import java.io.FileNotFoundException;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

```

```

class Song {
    private String title;

    public Song(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }
}

```

```

class Album {
    private String name;
    private List<Song> songs;

    public Album(String name) {

```

```

        this.name = name;
        this.songs = new ArrayList<>();
    }

    public void addSong(Song song) {
        songs.add(song);
    }

    public List<Song> getSongs() {
        return songs;
    }

    public String getName() {
        return name;
    }
}

class Artist {
    private String name;
    private Map<String, Album> albums;

    public Artist(String name) {
        this.name = name;
        this.albums = new HashMap<>();
    }

    public void addAlbum(Album album) {
        albums.put(album.getName(), album);
    }

    public Map<String, Album> getAlbums() {
        return albums;
    }

    public String getName() {
        return name;
    }
}

class Genre {
    private String name;
    private Map<String, Artist> artists;

    public Genre(String name) {
        this.name = name;
        this.artists = new HashMap<>();
    }
}

```

```

    }

    public void addArtist(Artist artist) {
        artists.put(artist.getName(), artist);
    }

    public Map<String, Artist> getArtists() {
        return artists;
    }

    public String getName() {
        return name;
    }

    public List<Artist> getSortedArtists() {
        List<Artist> artistList = new ArrayList<>(artists.values());
        Collections.sort(artistList, (a1, a2) -> a1.getName().compareToIgnoreCase(a2.getName()));
        return artistList;
    }
}

class SongQueue {
    private List<String> queue;

    public SongQueue() {
        queue = new ArrayList<>();
    }

    public void enqueue(String song) {
        queue.add(song);
    }

    public String dequeue() {
        if (!isEmpty()) {
            return queue.remove(0);
        }
        return null;
    }

    public boolean isEmpty() {
        return queue.isEmpty();
    }

    public String front() {
        if (!isEmpty()) {
            return queue.get(0);
        }
    }
}

```

```

    }
    return null;
}

public void display() {
    System.out.println("Queued Songs:");
    for (String song : queue) {
        System.out.println("- " + song);
    }
}
}

public class Create {
    private Map<String, Genre> musicLibrary;
    private List<String> playlist;
    private Scanner scanner;
    private static final String FILE_NAME = "playlists.txt";
    private SongQueue songQueue;

    public Create() {
        musicLibrary = new HashMap<>();
        playlist = new ArrayList<>();
        scanner = new Scanner(System.in);
        songQueue = new SongQueue();
        initializeLibrary();
    }

    private void initializeLibrary() {
        Genre rock = new Genre("Rock");

        Artist queen = new Artist("Queen");
        Album album1 = new Album("A Night at the Opera");
        album1.addSong(new Song("Bohemian Rhapsody"));
        album1.addSong(new Song("Love of My Life"));
        album1.addSong(new Song("You're My Best Friend"));
        album1.addSong(new Song("Killer Queen"));
        album1.addSong(new Song("Lazing on a Sunday Afternoon"));
        queen.addAlbum(album1);
        rock.addArtist(queen);

        Artist ledZeppelin = new Artist("Led Zeppelin");
        Album album2 = new Album("Led Zeppelin IV");
        album2.addSong(new Song("Stairway to Heaven"));
        album2.addSong(new Song("Black Dog"));
        album2.addSong(new Song("Rock and Roll"));
        album2.addSong(new Song("Going to California"));
    }
}

```



```
album2.addSong(new Song("When the Levee Breaks"));
ledZeppelin.addAlbum(album2);
rock.addArtist(ledZeppelin);
```

```
// Pop
```

```
Genre pop = new Genre("Pop");
```

```
Artist michaelJackson = new Artist("Michael Jackson");
Album album3 = new Album("Thriller");
album3.addSong(new Song("Thriller"));
album3.addSong(new Song("Beat It"));
album3.addSong(new Song("Billie Jean"));
album3.addSong(new Song("Human Nature"));
album3.addSong(new Song("P.Y.T. (Pretty Young Thing)"));
michaelJackson.addAlbum(album3);
pop.addArtist(michaelJackson);
```

```
Artist taylorSwift = new Artist("Taylor Swift");
Album album4 = new Album("1989");
album4.addSong(new Song("Shake It Off"));
album4.addSong(new Song("Blank Space"));
album4.addSong(new Song("Style"));
album4.addSong(new Song("Out of the Woods"));
album4.addSong(new Song("Bad Blood"));
taylorSwift.addAlbum(album4);
pop.addArtist(taylorSwift);
```

```
// Indie Genre
```

```
Genre indie = new Genre("Indie");
```

```
Artist theWallows = new Artist("The Wallows");
Album album5 = new Album("Nothing Matters");
album5.addSong(new Song("Are You Bored Yet? Ft. Clairo"));
album5.addSong(new Song("Sidelines"));
album5.addSong(new Song("Scrawny"));
album5.addSong(new Song("Do Not Wait"));
album5.addSong(new Song("Ice Cold Pool"));
theWallows.addAlbum(album5);
indie.addArtist(theWallows);
```

```
Artist bonIver = new Artist("Bon Iver");
Album album6 = new Album("22, A Million");
album6.addSong(new Song("22 (OVER S $\infty$ N)"));
album6.addSong(new Song("10 d E A T h b R E a s T"));
album6.addSong(new Song("33 'GOD'"));
album6.addSong(new Song("715 - CR $\Sigma\Sigma$ KS"));
```

```
album6.addSong(new Song("29 Strafford APTS"));
bonIver.addAlbum(album6);
indie.addArtist(bonIver);
```

```
// Rap
```

```
Genre rap = new Genre("Rap");
```

```
Artist kendrickLamar = new Artist("Kendrick Lamar");
Album album7 = new Album("DAMN.");
album7.addSong(new Song("BLOOD."));
album7.addSong(new Song("DNA."));
album7.addSong(new Song("HUMBLE."));
album7.addSong(new Song("LOYALTY. (feat. Rihanna)"));
album7.addSong(new Song("PRIDE."));
kendrickLamar.addAlbum(album7);
rap.addArtist(kendrickLamar);
```

```
Artist jCole = new Artist("J. Cole");
Album album8 = new Album("2014 Forest Hills Drive");
album8.addSong(new Song("Intro"));
album8.addSong(new Song("January 28th"));
album8.addSong(new Song("Wet Dream"));
album8.addSong(new Song("Fire Squad"));
album8.addSong(new Song("Love Yourz"));
jCole.addAlbum(album8);
rap.addArtist(jCole);
```

```
// HipHop
```

```
Genre hiphop = new Genre("HipHop");
```

```
Artist drake = new Artist("Drake");
Album album9 = new Album("Take Care");
album9.addSong(new Song("Marvins Room"));
album9.addSong(new Song("Headlines"));
album9.addSong(new Song("The Ride"));
album9.addSong(new Song("Take Care (feat. Rihanna)"));
album9.addSong(new Song("Lord Knows (feat. Rick Ross)"));
drake.addAlbum(album9);
hiphop.addArtist(drake);
```

```
Artist lilWayne = new Artist("Lil Wayne");
Album album10 = new Album("Tha Carter III");
album10.addSong(new Song("3 Peat"));
album10.addSong(new Song("Mr. Carter (feat. Jay-Z)"));
album10.addSong(new Song("A Milli"));
album10.addSong(new Song("Got Money (feat. T-Pain)"));
```

```
album10.addSong(new Song("Lollipop (feat. Static Major)"));
lilWayne.addAlbum(album10);
hiphop.addArtist(lilWayne);
```

```
// Jazz
```

```
Genre jazz = new Genre("Jazz");
```

```
Artist milesDavis = new Artist("Miles Davis");
Album album11 = new Album("Kind of Blue");
album11.addSong(new Song("So What"));
album11.addSong(new Song("Freddie Freeloader"));
album11.addSong(new Song("Blue in Green"));
album11.addSong(new Song("All Blues"));
album11.addSong(new Song("Flamenco Sketches"));
milesDavis.addAlbum(album11);
jazz.addArtist(milesDavis);
```

```
Artist johnColtrane = new Artist("John Coltrane");
Album album12 = new Album("A Love Supreme");
album12.addSong(new Song("Acknowledgement"));
album12.addSong(new Song("Resolution"));
album12.addSong(new Song("Pursuance"));
album12.addSong(new Song("Psalm"));
album12.addSong(new Song("A Love Supreme, Pt. 1"));
johnColtrane.addAlbum(album12);
jazz.addArtist(johnColtrane);
```

```
// KPop
```

```
Genre kpop = new Genre("KPop");
```

```
Artist bts = new Artist("BTS");
Album album13 = new Album("Map of the Soul: 7");
album13.addSong(new Song("Intro: Persona"));
album13.addSong(new Song("Boy With Luv (feat. Halsey)"));
album13.addSong(new Song("Fake Love"));
album13.addSong(new Song("ON"));
album13.addSong(new Song("Black Swan"));
bts.addAlbum(album13);
kpop.addArtist(bts);
```

```
Artist blackpink = new Artist("BLACKPINK");
Album album14 = new Album("The Album");
album14.addSong(new Song("How You Like That"));
album14.addSong(new Song("Ice Cream (with Selena Gomez)"));
album14.addSong(new Song("Lovesick Girls"));
album14.addSong(new Song("Pretty Savage"));
```

```

album14.addSong(new Song("Bet You Wanna (feat. Cardi B)"));
blackpink.addAlbum(album14);
kpop.addArtist(blackpink);

musicLibrary.put("Rock", rock);
musicLibrary.put("Pop", pop);
musicLibrary.put("Indie", indie);
musicLibrary.put("Rap", rap);
musicLibrary.put("HipHop", hiphop);
musicLibrary.put("Jazz", jazz);
musicLibrary.put("KPop", kpop);
}
public void createPlaylist() {
    System.out.print("Enter the name of your playlist: ");
    String playlistName = scanner.nextLine();

    while (true) {
        List<Genre> genres = new ArrayList<>(musicLibrary.values());
        System.out.println("\nAvailable Genres:");
        for (int i = 0; i < genres.size(); i++) {
            System.out.println((i + 1) + ". " + genres.get(i).getName());
        }
        System.out.println("0. Done");

        System.out.print("Choose a genre (number): ");
        int genreIndex = getValidInput(0, genres.size());
        if (genreIndex == 0) break;

        Genre selectedGenre = genres.get(genreIndex - 1);
        List<Artist> artists = new ArrayList<>(selectedGenre.getArtists().values());

        while (true) {
            System.out.println("\nArtists in " + selectedGenre.getName() + ":");
            for (int i = 0; i < artists.size(); i++) {
                System.out.println((i + 1) + ". " + artists.get(i).getName());
            }
            System.out.println("0. Back");

            System.out.print("Choose an artist (number): ");
            int artistIndex = getValidInput(0, artists.size());
            if (artistIndex == 0) break;

            Artist selectedArtist = artists.get(artistIndex - 1);
            List<Album> albums = new ArrayList<>(selectedArtist.getAlbums().values());

```

```

while (true) {
    System.out.println("\nAlbums by " + selectedArtist.getName() + ":");
    for (int i = 0; i < albums.size(); i++) {
        System.out.println((i + 1) + ". " + albums.get(i).getName());
    }
    System.out.println("0. Back");

    System.out.print("Choose an album (number): ");
    int albumIndex = getValidInput(0, albums.size());
    if (albumIndex == 0) break;

    Album selectedAlbum = albums.get(albumIndex - 1);
    List<Song> songs = selectedAlbum.getSongs();

    while (true) {
        System.out.println("\nSongs in " + selectedAlbum.getName() + ":");
        for (int i = 0; i < songs.size(); i++) {
            System.out.println((i + 1) + ". " + songs.get(i).getTitle());
        }
        System.out.println("0. Back");

        System.out.print("Choose a song to add (number): ");
        int songIndex = getValidInput(0, songs.size());
        if (songIndex == 0) break;

        Song selectedSong = songs.get(songIndex - 1);
        playlist.add(selectedSong.getTitle());
        System.out.println("'" + selectedSong.getTitle() + "' added to your playlist!");
    }
}

// Save the playlist only if it's not empty
if (playlist.isEmpty()) {
    System.out.println("No songs in the playlist to save.");
} else {
    savePlaylistToFile(playlistName);
    System.out.println("\nPlaylist '" + playlistName + "' created successfully!");
}

private void savePlaylistToFile(String playlistName) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(FILE_NAME, true))) { //

```

Append mode

```

        writer.write("Playlist: " + playlistName);
        writer.newLine();
        writer.write("=====");
        writer.newLine();

        for (String song : playlist) {
            writer.write(song);
            writer.newLine();
        }

        writer.write("-----");
        writer.newLine();

        // Ensure everything is written
        writer.flush();
        System.out.println("Playlist saved to: " + FILE_NAME);
    } catch (IOException e) {
        System.out.println("An error occurred while saving the playlist.");
        e.printStackTrace();
    }
}

public void clearPlaylist() {
    playlist.clear();
    System.out.println("Current playlist cleared.");
}

public void removeSongFromPlaylist() {
    if (playlist.isEmpty()) {
        System.out.println("Your playlist is empty.");
        return;
    }

    System.out.println("Current Playlist:");
    for (int i = 0; i < playlist.size(); i++) {
        System.out.println((i + 1) + ". " + playlist.get(i));
    }

    System.out.print("Enter the number of the song to remove: ");
    int songIndex = getValidInput(1, playlist.size()) - 1; // Adjust for zero-based index
    String removedSong = playlist.remove(songIndex);
    System.out.println("'" + removedSong + "' has been removed from your playlist.");
}

public void queueSong() {

```

```

System.out.println("Select a song to queue.");
for (int i = 0; i < playlist.size(); i++) {
    System.out.println((i + 1) + ". " + playlist.get(i));
}
System.out.print("Enter the number of the song to queue: ");
int songIndex = getValidInput(1, playlist.size()) - 1; // Adjust for zero-based index
String songToQueue = playlist.get(songIndex);
songQueue.enqueue(songToQueue);
System.out.println(""" + songToQueue + "" has been added to the queue.");
}

public void playNextSong() {
    if (songQueue.isEmpty()) {
        System.out.println("No songs in the queue to play.");
        return;
    }
    String songToPlay = songQueue.dequeue();
    System.out.println("Now playing: " + songToPlay);
}

public void showQueue() {
    songQueue.display();
}

public List<String> getPlaylist() {
    return new ArrayList<>(playlist); // Return a copy of the playlist
}

public void viewPlaylists() {
    try (Scanner fileScanner = new Scanner(new File(FILE_NAME))) {
        System.out.println("\nExisting Playlists:");
        while (fileScanner.hasNextLine()) {
            System.out.println(fileScanner.nextLine());
        }
    } catch (FileNotFoundException e) {
        System.out.println("No playlists found.");
    }
}

private int getValidInput(int min, int max) {
    int choice;
    while (true) {
        try {
            choice = Integer.parseInt(scanner.nextLine());
            if (choice >= min && choice <= max) {
                return choice;
            } else {

```

```

        System.out.print("Invalid choice, try again: ");
    }
    } catch (NumberFormatException e) {
        System.out.print("Invalid input, enter a number: ");
    }
    }
}

public Map<String, Genre> getMusicLibrary() {
    return musicLibrary; // musiclibrary access for the generate class - lets it so i dont have to rewrite it
}
}

package Music;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class View {
    private static final String FILE_NAME = "playlists.txt";

    // Method to load playlists from the file into a 2D array
    public static String[][] loadPlaylists() {
        File file = new File(FILE_NAME);

        if (!file.exists() || file.length() == 0) {
            System.out.println("No playlists found.");
            return new String[0][0]; // Return an empty 2D array
        }

        List<String[]> playlists = new ArrayList<>();

        try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
            String line;
            String[] currentPlaylist = null;
            int songCount = 0;

            while ((line = reader.readLine()) != null) {
                if (line.startsWith("Playlist:")) {
                    if (currentPlaylist != null) {
                        playlists.add(currentPlaylist);
                    }
                    // Start a new playlist
                    currentPlaylist = new String[10]; // Assume a maximum of 10 songs per playlist
                    songCount = 0; // Reset song count for the new playlist
                }
            }
        }
    }
}

```



```

        continue;
    } else if (line.startsWith("-----")) {
        // End of the current playlist
        if (currentPlaylist != null) {
            playlists.add(currentPlaylist);
            currentPlaylist = null; // Reset for the next playlist
        }
        continue;
    }

    // Add song to the current playlist
    if (currentPlaylist != null) {
        if (songCount < currentPlaylist.length) {
            currentPlaylist[songCount++] = line; // Add song to the playlist
        }
    }
}

// Add the last playlist if it exists
if (currentPlaylist != null) {
    playlists.add(currentPlaylist);
}

} catch (IOException e) {
    System.out.println("An error occurred while reading the file.");
    e.printStackTrace();
}

// List<String[]> to String[][]
String[][] result = new String[playlists.size()][];
for (int i = 0; i < playlists.size(); i++) {
    result[i] = playlists.get(i);
}

return result;
}

// display the playlists
public static void displayPlaylists() {
    String[][] playlists = loadPlaylists();

    if (playlists.length == 0) {
        System.out.println("No playlists found.");
        return;
    }
}

```

```

System.out.println("\n=== Your Saved Playlists ===\n");

for (int i = 0; i < playlists.length; i++) {
    System.out.println("Playlist " + (i + 1) + ":");
    for (String song : playlists[i]) {
        if (song != null) { // Check for null to not print any empty slots
            System.out.println(" - " + song);
        }
    }
    System.out.println("-----");
}

System.out.println("\n=====");
}
}

```